HabitatHero

A PERSONALIZED HDB RECOMMENDER

Prepared by TCE2, Group 2:
NG CAI XIAN
CHONG XIN LE
CHOW ZEN TENG
SIM YONG EN, IVAN
YEONG JIA WEN, JONATHAN
MUHAMMAD IZZUDDIN BIN HUSEIN

A G E N D A

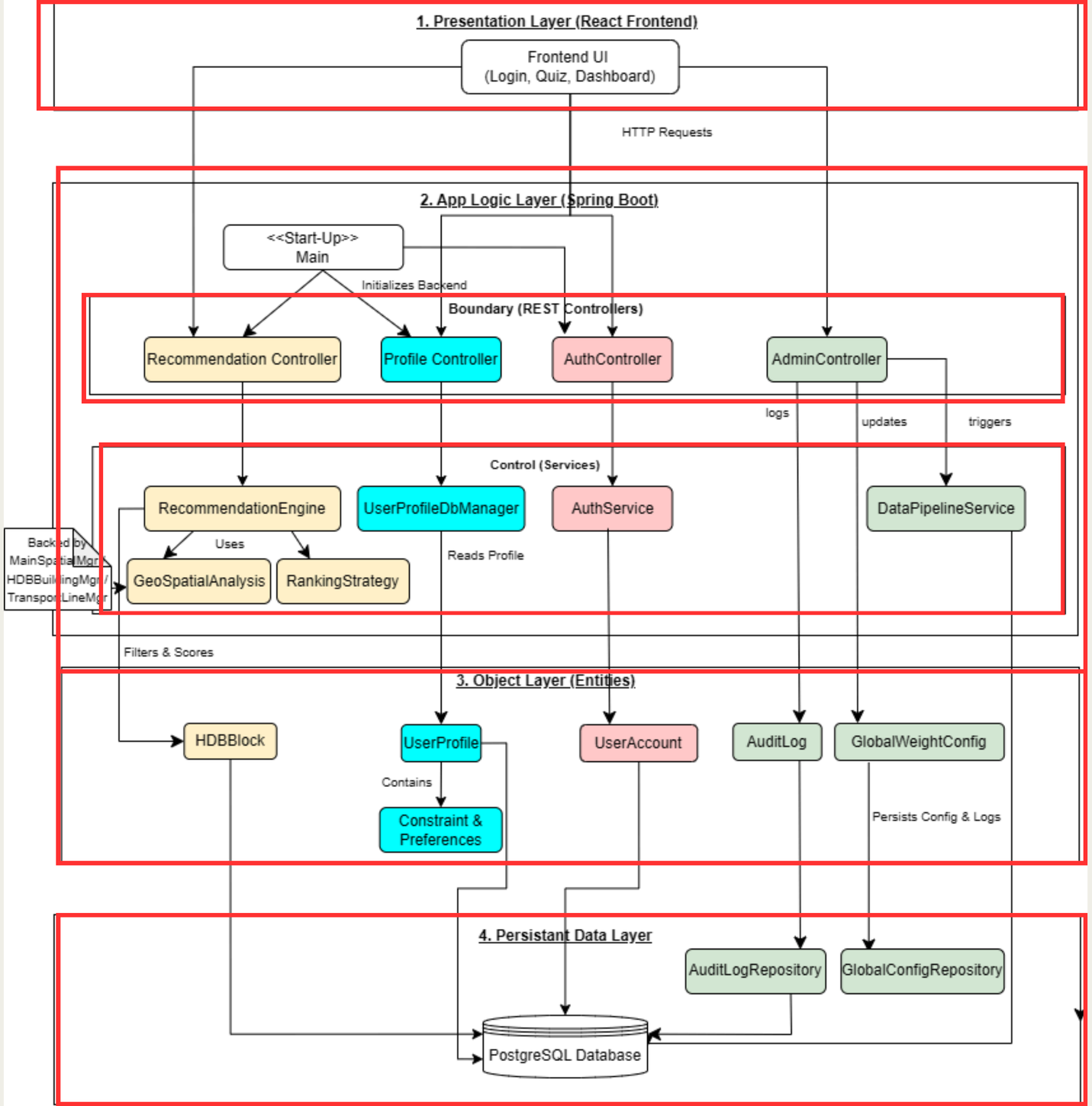
- Motivation
- SWE Practices
  - System Architecture
  - Design Patterns
- Traceability
  - Use Case
  - Class Diagram
  - Implementation
- Live Demo
  - Persona 1
  - Persona 2
  - Persona 3
- System Robustness & Compliance

OUR MOTIVATION

To generate personalized HDB matches by filtering, analyzing and ranking HDBs based on:

- Key constraints (Location, Flat type, Budget)
- Commute needs
- Unique livability factors such as solar orientation, noise levels
- Flags future development risk

SYSTEM ARCHITECTURE



HOW LAYERS COMMUNICATE: IOC, DI, AND REPOSITORIES

```java
@Service        You, last month • Implemented Spring Boot REST API for Recommenda…
public class RecommendationEngine {

    private static final String FACTOR_SOLAR = "Solar Orientation";
    private static final String FACTOR_ACOUSTIC = "Acoustic Comfort";
    private static final String FACTOR_CONVENIENCE = "Convenience";
    private static final double STRICT_NOISE_DISTANCE_METERS = 100.0;
    private static final double STRICT_CONVENIENCE_MIN_SCORE = 0.5;
    private static final double QUIET_NOISE_DB = 55.0;
    private static final double LOUD_NOISE_DB = 85.0;

    private final ResaleTransactionRepository resaleTransactionRepository;
    private final MultiCommuterService multiCommuterService;
    private final HDBBuildingMgr hdbBuildingMgr;
    private final TransportLineMgr transportLineMgr;
    private final ConvenienceScoringService convenienceScoringService;
    private final RankingStrategy rankingStrategy;

    public RecommendationEngine(ResaleTransactionRepository resaleTransactionRepository,
                                MultiCommuterService multiCommuterService,
                                HDBBuildingMgr hdbBuildingMgr,
                                TransportLineMgr transportLineMgr,
                                ConvenienceScoringService convenienceScoringService,
                                RankingStrategy rankingStrategy) {
        this.resaleTransactionRepository = resaleTransactionRepository;
        this.multiCommuterService = multiCommuterService;
        this.hdbBuildingMgr = hdbBuildingMgr;
        this.transportLineMgr = transportLineMgr;
        this.convenienceScoringService = convenienceScoringService;
        this.rankingStrategy = rankingStrategy;
    }
```

```java
private Map<String, ResaleTransactionRepository.TownPsfView>
loadTownPriceBenchmarks(List<BlockCandidateView> candidateList) {
    List<String> towns = candidateList.stream()
            .map(candidate -> candidate.getBlock() ==
            null ? null : candidate.getBlock().getTown())
            .map(this::normalizeTownKey)
            .filter(town -> town != null && !town.isBlank())
            .distinct()
            .collect(Collectors.toList());

    if (towns.isEmpty()) {
        return Map.of();
    }

    return resaleTransactionRepository.findAveragePsfByTowns(towns).stream()
        .filter(view -> view.getTown() != null && !view.getTown().isBlank())
        .collect(Collectors.toMap(      You, 6 hours ago • Uncommitted chang
                view -> normalizeTownKey(view.getTown()),
                view -> view,
                (left, right) -> left,
                LinkedHashMap::new));
}
```

HOW LAYERS COMMUNICATE: IOC, DI, AND REPOSITORIES

```java
@Repository
public interface ResaleTransactionRepository extends
JpaRepository<ResaleTransaction, Long> {

    xinle, 5 days ago | 1 author (xinle)
    interface BlockFloorAreaView {
        Integer getBlockId();
        Double getAvgFloorAreaSqm();
    }

    xinle, 5 days ago | 1 author (xinle)
    interface TownPsfView {
        String getTown();
        Double getAvgPsf();
        Long getTransactionCount();
    }

    @Query("""
        SELECT new habitathero.control.BlockCandidateView(
            b,
            AVG(t.resalePrice),
            AVG(t.floorAreaSqm),
            AVG(t.remainingLease),
            COUNT(t)
        )
        FROM ResaleTransaction t
        JOIN t.block b
        WHERE b.coordinates IS NOT NULL
        AND (:townFilterDisabled = true OR UPPER(TRIM(b.town)) IN :preferredTowns)
        AND (:preferredFlatType IS NULL OR :preferredFlatType = '' OR
        UPPER(TRIM(t.flatType)) = UPPER(TRIM(:preferredFlatType)))
        GROUP BY b
        HAVING AVG(t.resalePrice) <= :maxBudget
        AND AVG(t.remainingLease) >= :minLeaseYears
        """)
```

```java
List<BlockFloorAreaView> findAverageFloorAreaByBlockIds(@Param("blockIds")
List<Integer> blockIds);

@Query("""
    SELECT
        UPPER(TRIM(t.town)) AS town,
        AVG(t.resalePrice / (t.floorAreaSqm * 10.7639))
        AS avgPsf,
        COUNT(t) AS transactionCount
    FROM ResaleTransaction t
    WHERE UPPER(TRIM(t.town)) IN :towns
    AND t.floorAreaSqm > 0
    GROUP BY UPPER(TRIM(t.town))
    """)
List<TownPsfView> findAveragePsfByTowns
(@Param("towns") List<String> towns);
```

DESIGN PATTERN: STRATEGY

```java
private final RankingStrategy rankingStrategy;      You, 6 hours ago • Changed name

public RecommendationEngine(ResaleTransactionRepository resaleTransactionRepository,
                            MultiCommuterService multiCommuterService,
                            HDBBuildingMgr hdbBuildingMgr,
                            TransportLineMgr transportLineMgr,
                            ConvenienceScoringService convenienceScoringService,
                            RankingStrategy rankingStrategy) {
    this.resaleTransactionRepository = resaleTransactionRepository;
    this.multiCommuterService = multiCommuterService;
    this.hdbBuildingMgr = hdbBuildingMgr;
    this.transportLineMgr = transportLineMgr;
    this.convenienceScoringService = convenienceScoringService;
    this.rankingStrategy = rankingStrategy;
```

```java
    return rankingStrategy.rank(rankedBlocks, request);
}
```

```java
@Component
public class GlobalMatchRankingStrategy implements RankingStrategy {

    @Override
    public List<HDBBlock> rank(List<HDBBlock> candidateBlocks, UserProfile profile) {
        if (candidateBlocks == null || candidateBlocks.isEmpty()) {
            return candidateBlocks == null ? List.of() : candidateBlocks;
        }

        List<HDBBlock> ranked = new ArrayList<>(candidateBlocks);
        ranked.sort(
                Comparator.comparingDouble(HDBBlock::getGlobalMatchIndex).reversed()
                .thenComparingDouble(HDBBlock::getEstimatedPrice)
                .thenComparing(Comparator.comparingInt
                (HDBBlock::getRemainingLeaseYears).reversed()));
        return ranked;
```

DESIGN PATTERN: FACADE

```java
private static MainSpatialMgr instance;
private HDBBuildingMgr hdbBuildingMgr;
private TransportLineMgr transportLineMgr;
private LandUseMgr landUseMgr;
/**
Initializes manager dependencies used by the spatial facade */
private MainSpatialMgr() {
    this.hdbBuildingMgr = HDBBuildingMgr.getInstance();
    this.transportLineMgr = TransportLineMgr.getInstance();
    this.landUseMgr = LandUseMgr.getInstance();
}
/**
 * Returns the singleton instance of MainSpatialMgr.
 *
 * @return singleton MainSpatialMgr instance*/
public static MainSpatialMgr getInstance() {
    if (instance == null) {
        instance = new MainSpatialMgr();}
    return instance;}
```

```java
/** Returns transport noise analysis using a caller-specified radius. */
public JSONObject getNoiseLevel(String postalCode, double radius) {
    return transportLineMgr.getNoiseLevel(postalCode, radius);}


/** Sun-facing wrappers: default, custom azimuth, explicit east/west, and fast mode. */
public JSONObject getSunFacing(String postal_code)
{ return hdbBuildingMgr.getSunFacing(postal_code); }
public JSONObject getSunFacing(String postal_code, double customAzimuth)
{ return hdbBuildingMgr.getSunFacing(postal_code, customAzimuth); }
public JSONObject getSunFacing(String postalCode, double eastAzimuth, double westAzimuth)
{ return hdbBuildingMgr.getSunFacing(postalCode, eastAzimuth, westAzimuth); }
public JSONObject getSunFacingFast(String postalCode, double fullSweepStepDegrees,
double dayArcStepDegrees) { return
hdbBuildingMgr.getSunFacingFast(postalCode, fullSweepStepDegrees, dayArcStepDegrees); }
```

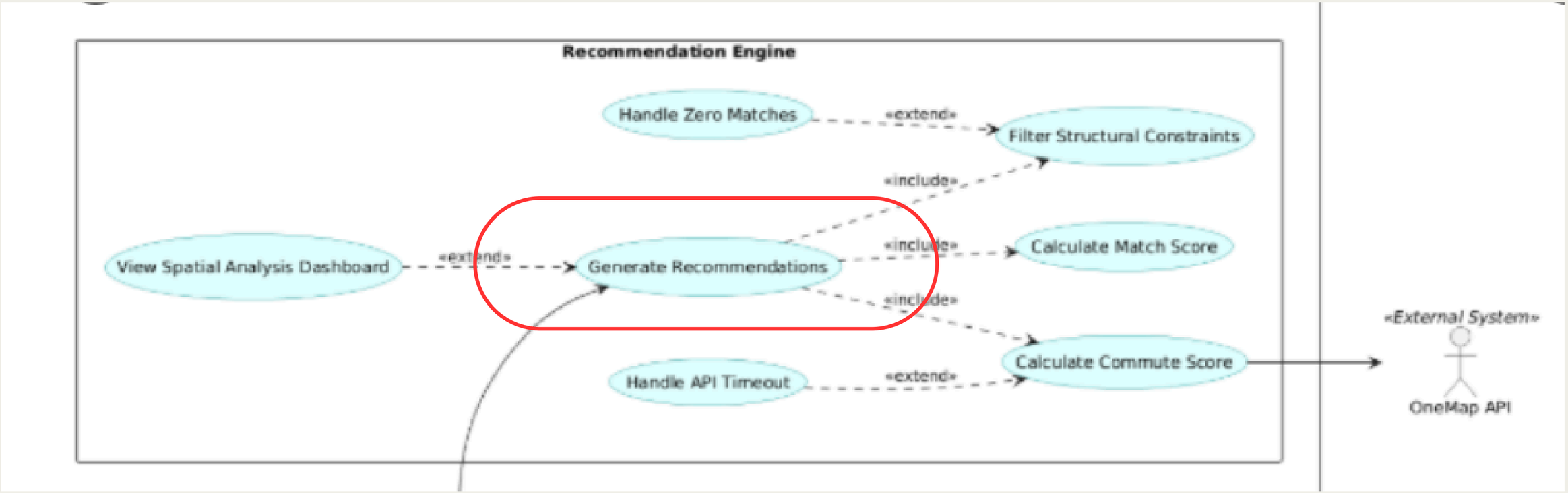
TRACEBILITY:

Use Case Diagram

method: calSunFacing()

Algorithm and calculation for sun-facing analysis

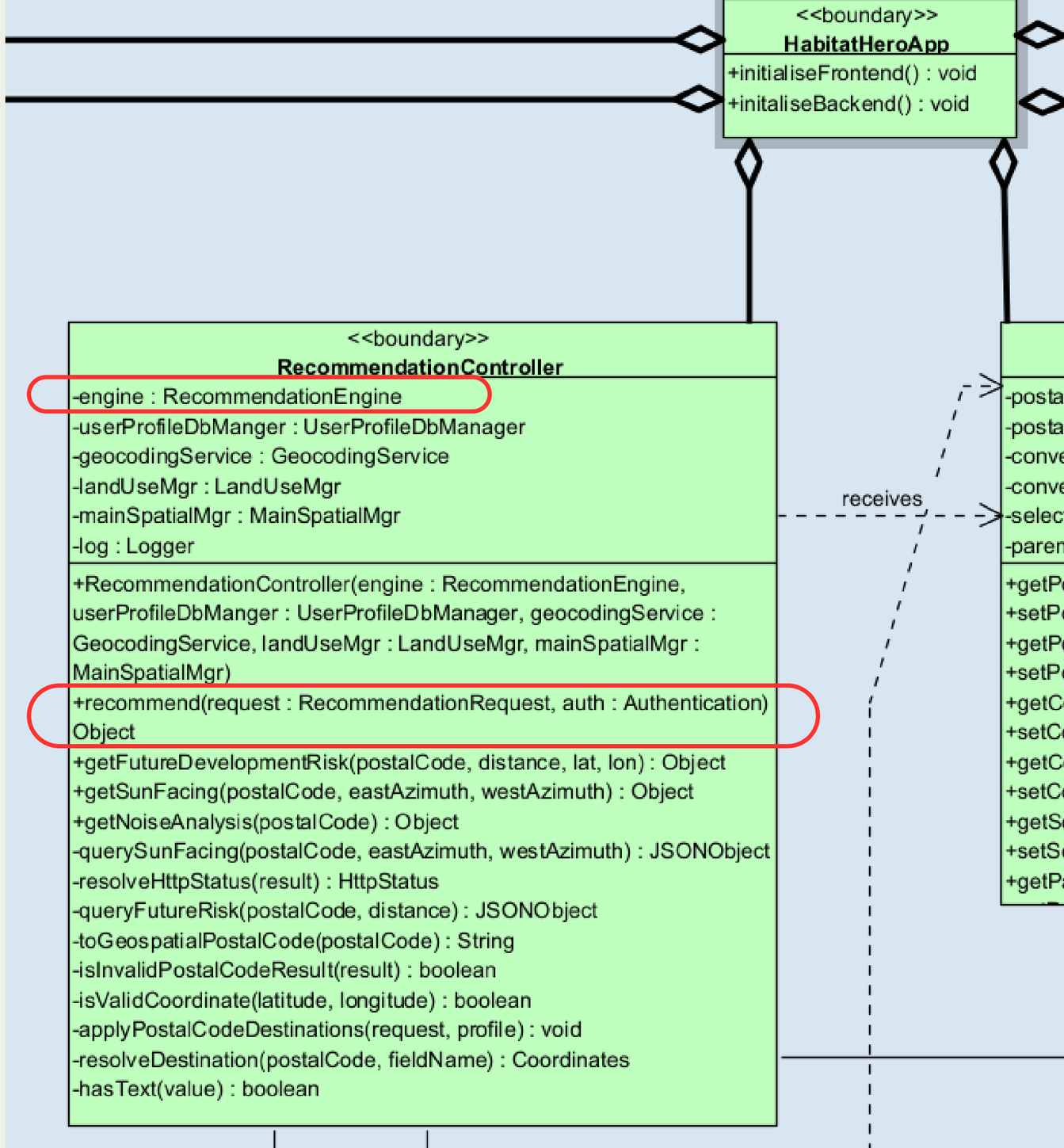
USE CASE DIAGRAM: GENERATE RECOMMENDATION



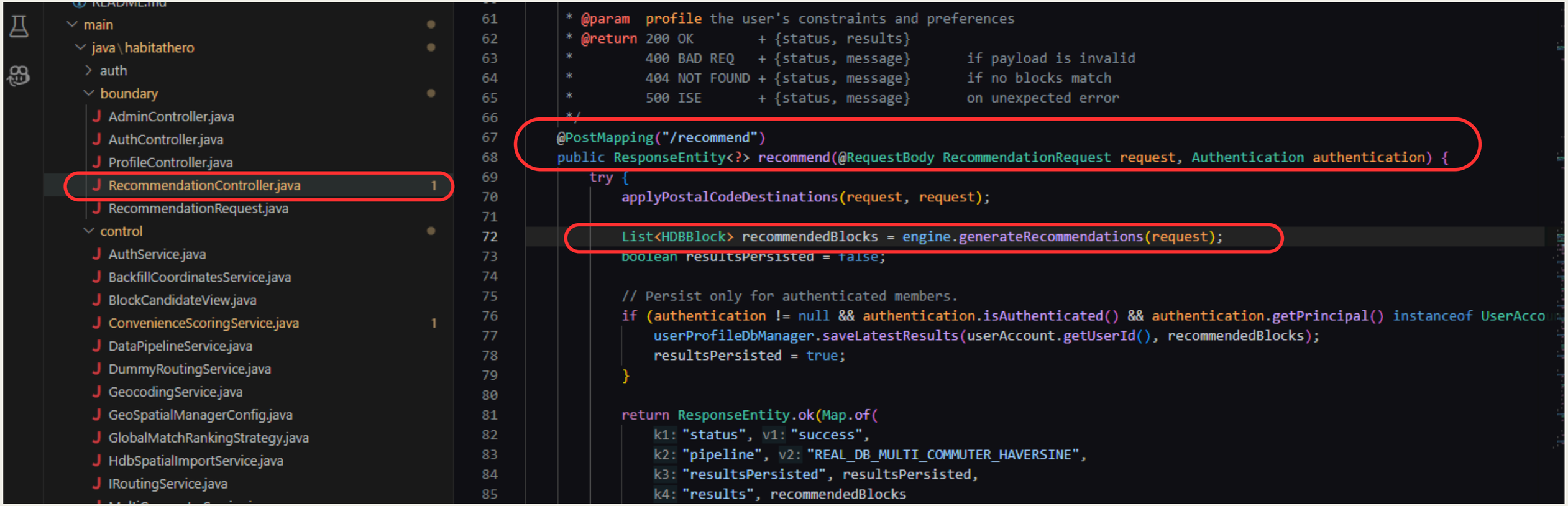
USE CASE DESCRIPTION: GENERATE RECOMMENDATION

Use Case ID:	UC-04		
Use Case Name:	Generate Recommendations		
Created By:	Habitat Hero Team	Last Updated By:	Izzuddin
Date Created:	Feb 5, 2026	Date Last Updated:	Apr 12, 2026
Actor:	User		
Description:	The system filters properties and ranks them based on a weighted "Match Percentage" (Lifestyle + Commute).		
Preconditions:	1. System is up and running. 2. User has a valid internet connection. 3. User has completed the quiz or User has previously saved results in profile storage.		
Postconditions:	1. Recommendation results are displayed to the user. 2. Results are either temporarily cached or loaded from/saved to profile storage depending on authentication state.		
Priority:	High		
Frequency of Use:	High (Triggered immediately after UC-01)		
Flow of Events:	1. The user accesses the Results page after submitting the housing preference quiz or retrieving saved recommendation results. 2. The system loads the available ranked HDB block results. 3. The system displays the user's submitted preference summary. 4. The system displays the ranked HDB block recommendations. 5. The user reviews recommendation details such as match score, estimated price, remaining lease, and floor area where available. 6. The user may search the displayed results. 7. The user may sort the displayed results using available sort options. 8. The user may browse the results across multiple pages. 9. The user may select to edit preferences and return to the quiz workflow. 10. The system preserves temporary guest results for the active session or loads saved member results where available.		

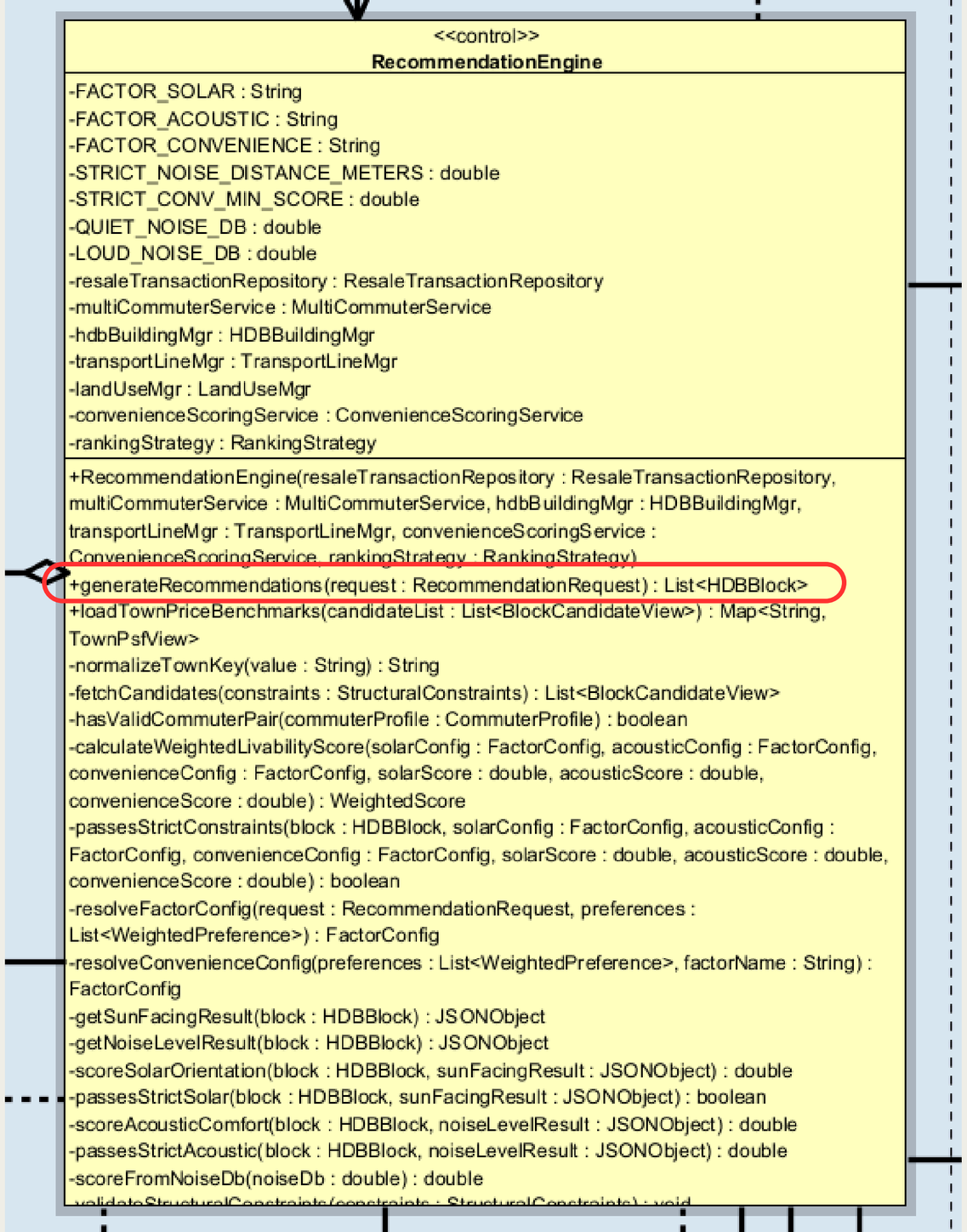
CLASS DIAGRAM: RECOMMENDATION CONTROLLER



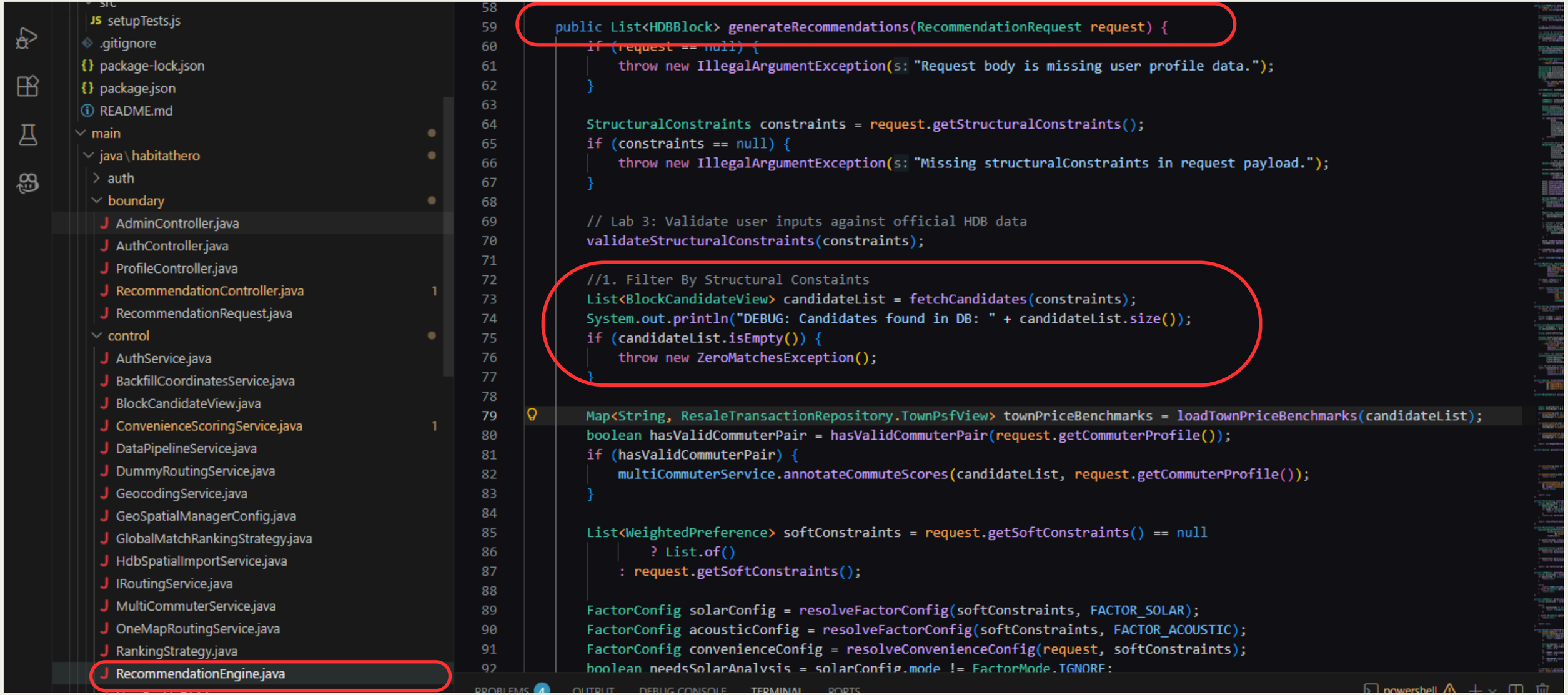
CODE: RECOMMENDATION CONTROLLER



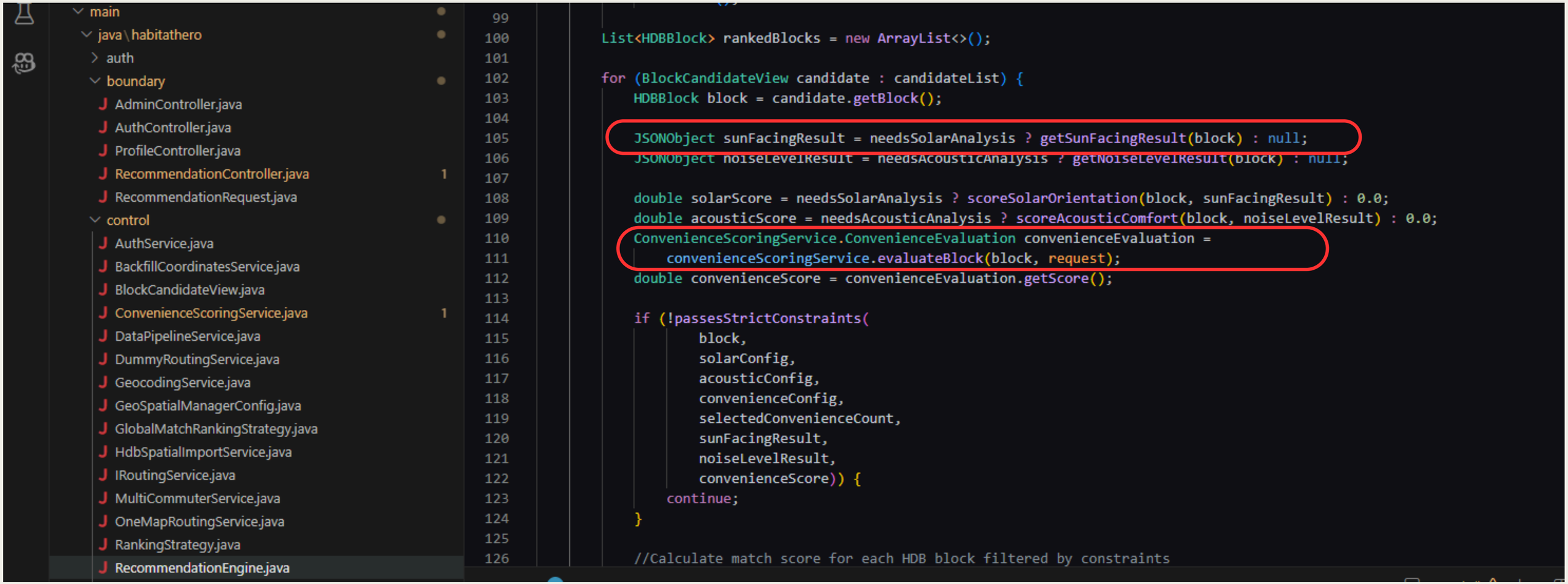
CLASS DIAGRAM: RECOMMENDATION ENGINE



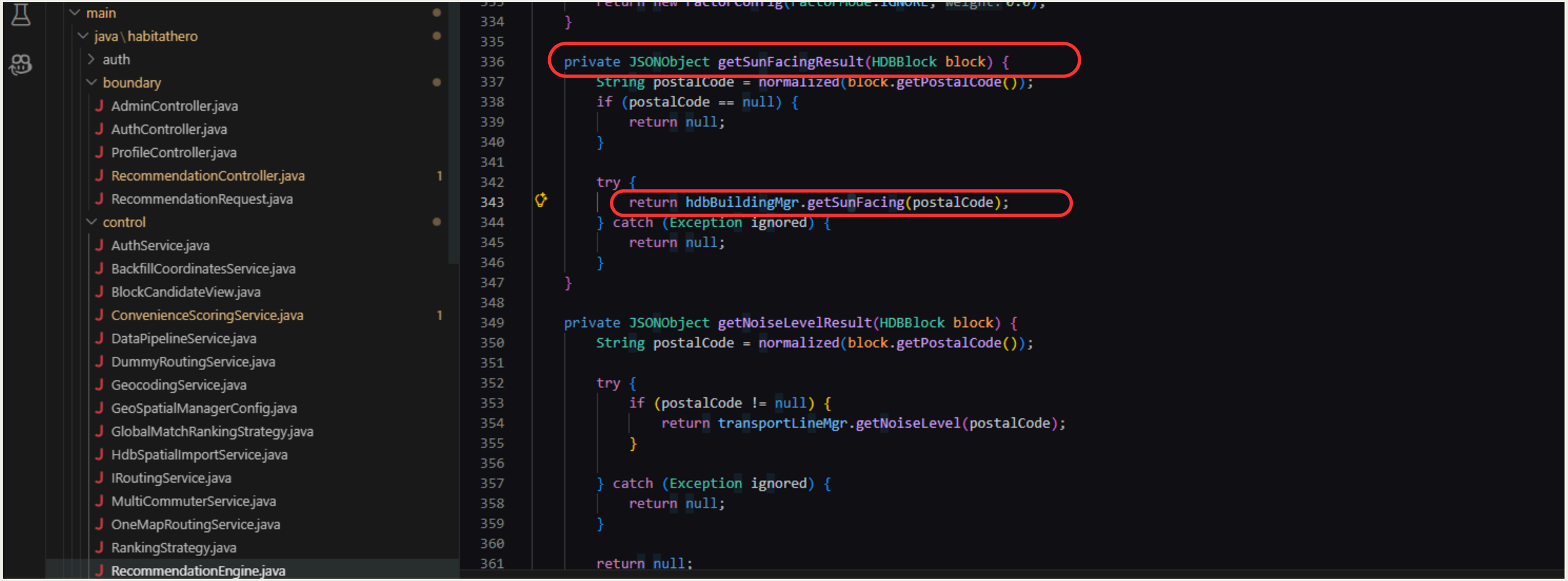
CODE: RECOMMENDATION ENGINE



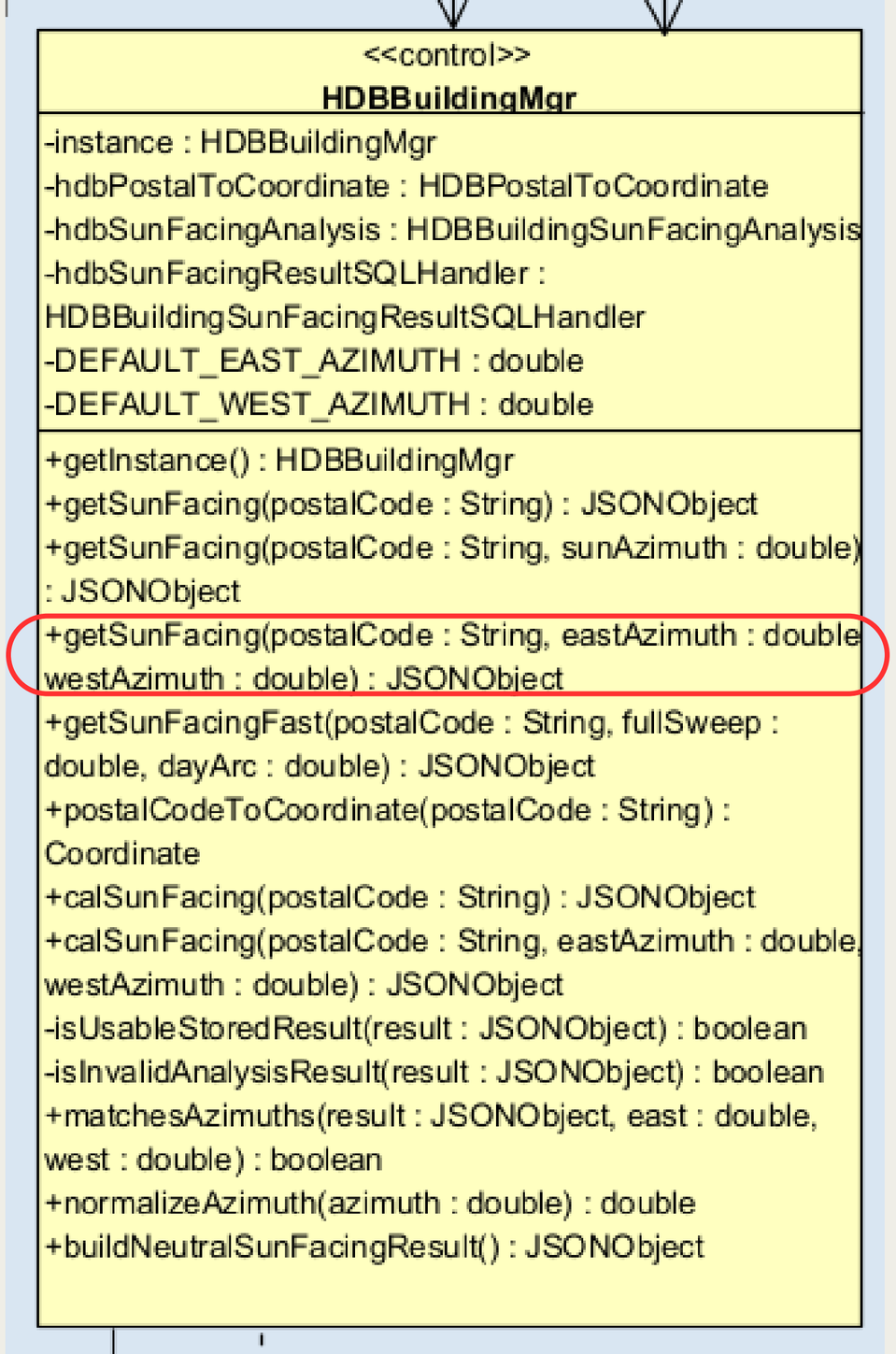
CODE: RECOMMENDATION ENGINE



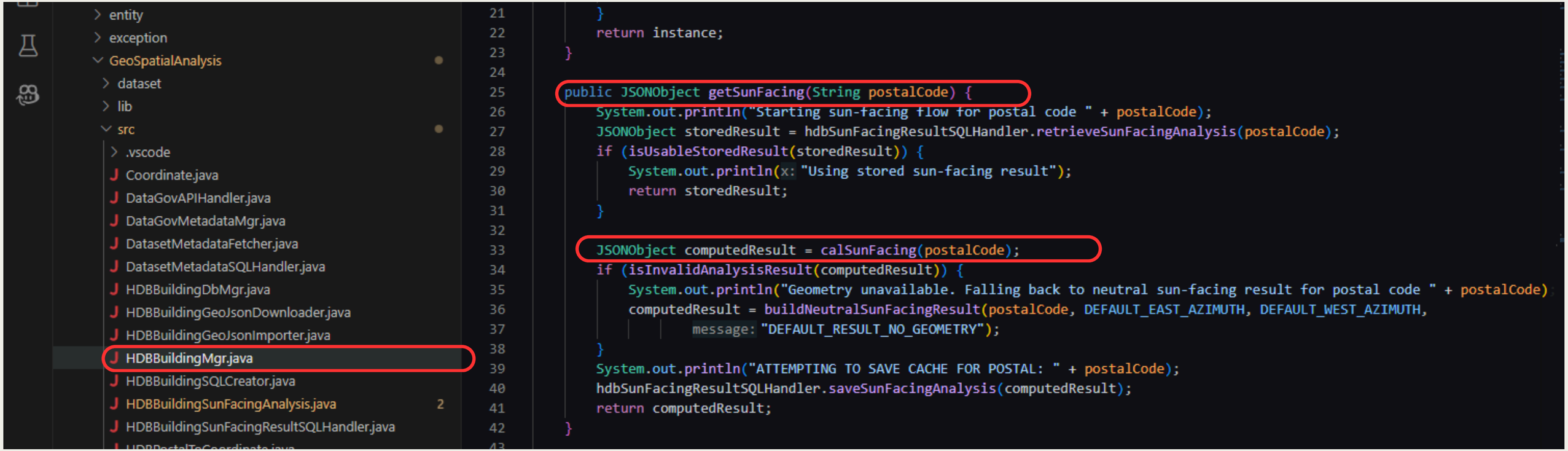
CODE: RECOMMENDATION ENGINE



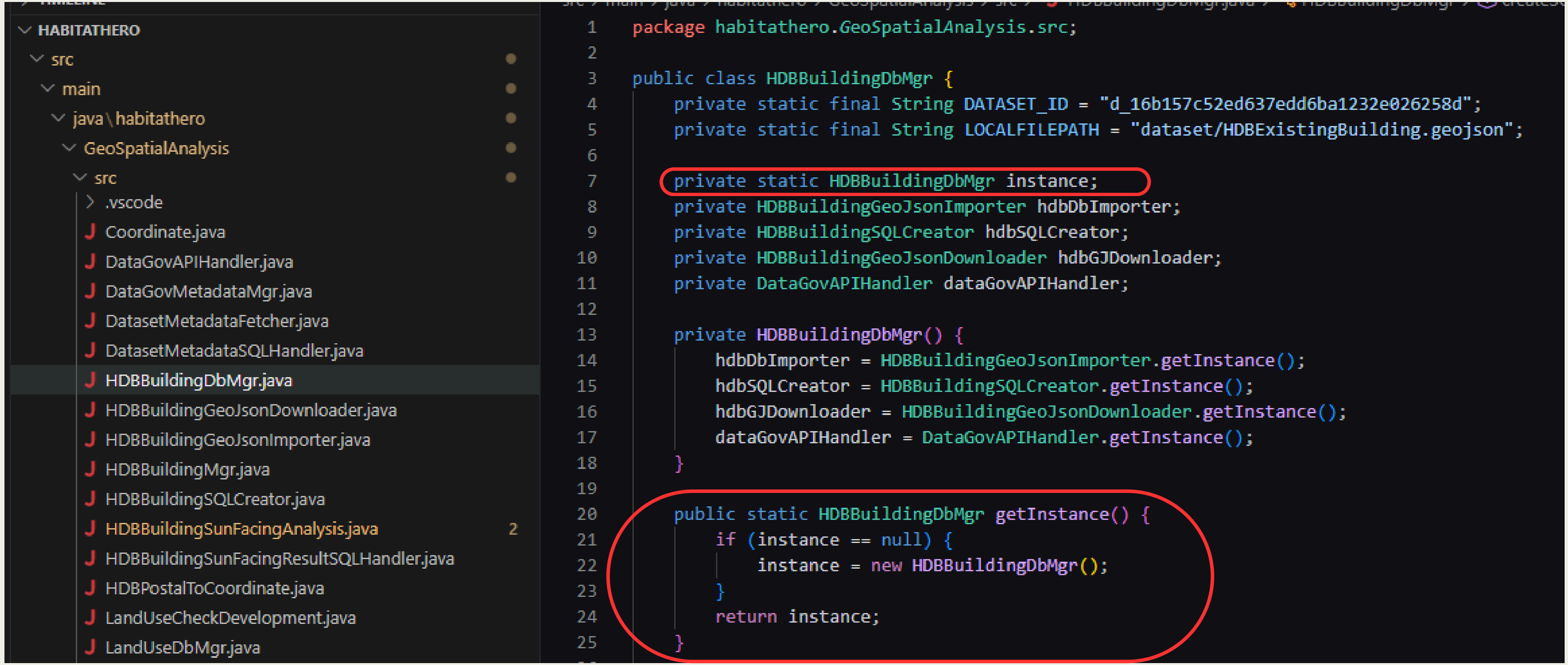
CLASS DIAGRAM: HDBBUILDINGMGR



CODE: HDBBUILDINGMGR

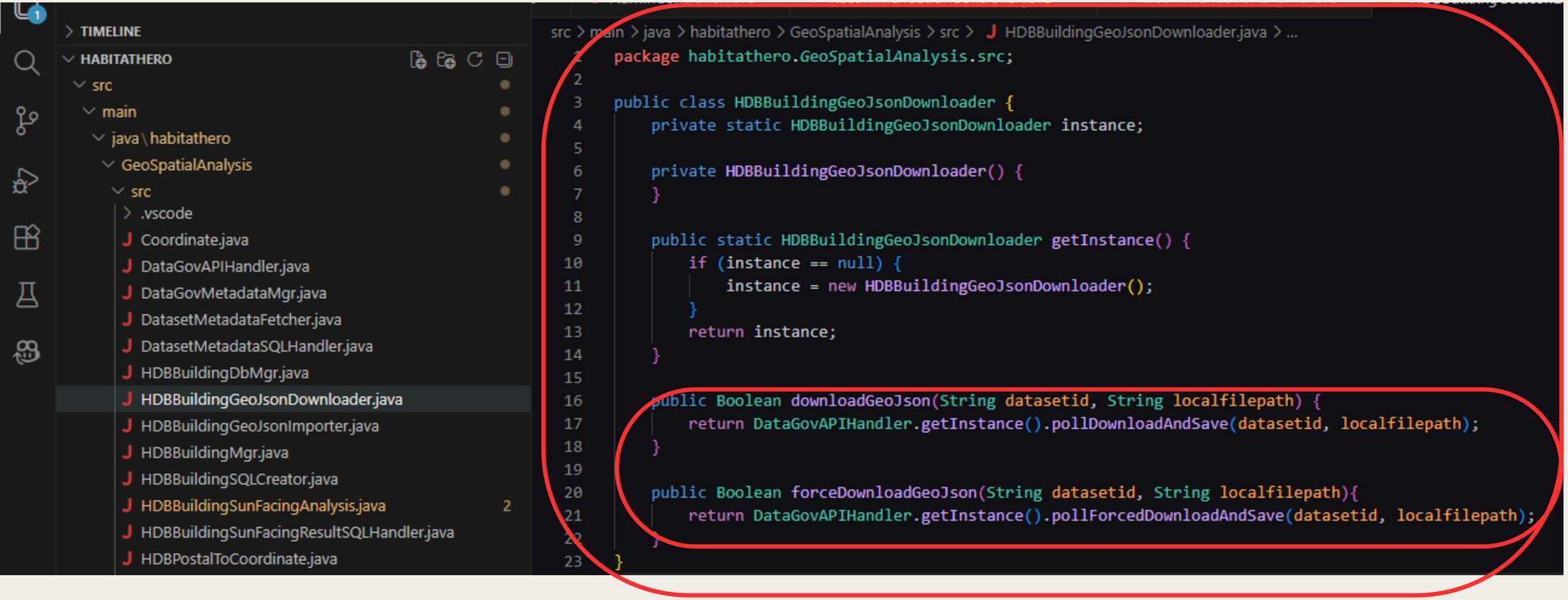


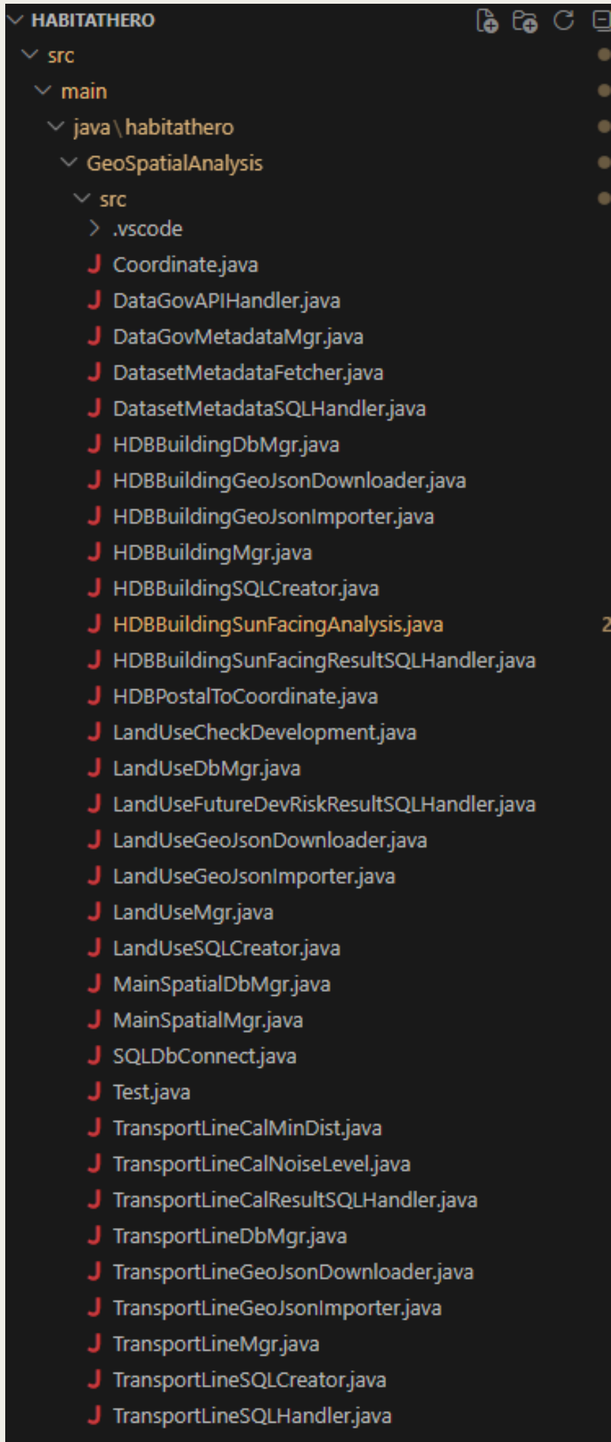
DESIGN IMPLEMENTAITON: SINGLETON



DESIGN IMPLEMENTATION:

SINGLE RESPONSIBILITY PRINCIPLE(SRP)





TESTING: SRP

TestSingleton.java

src > test > java > habitathero > GeoSpatialAnalysis > src > TestSingleton.java > { } habitathero.GeoSpatialAnalysis.src

```java
package habitathero.GeoSpatialAnalysis.src;

import static org.junit.jupiter.api.Assertions.assertSame;

import java.util.function.Supplier;
import java.util.stream.Stream;

import org.junit.jupiter.params.ParameterizedTest;
import org.junit.jupiter.params.provider.Arguments;
import org.junit.jupiter.params.provider.MethodSource;

class TestSingleton {

    @ParameterizedTest(name = "{0}")
    @MethodSource("singletonFactories")
    void singletonShouldReturnTheSameInstance(String singletonName, Supplier<Object> singletonFactory) {
        assertSame(singletonFactory.get(), singletonFactory.get(), singletonName + " should be a singleton");
    }

    private static Stream<Arguments> singletonFactories() {
        return Stream.of(
                Arguments.of("DataGovAPIHandler", (Supplier<Object>) DataGovAPIHandler::getInstance),
                Arguments.of("DataGovMetadataMgr", (Supplier<Object>) DataGovMetadataMgr::getInstance),
                Arguments.of("DatasetMetadataFetcher", (Supplier<Object>) DatasetMetadataFetcher::getInstance),
                Arguments.of("DatasetMetadataSQLHandler", (Supplier<Object>) DatasetMetadataSQLHandler::getInstance),
                Arguments.of("HDBBuildingDbMgr", (Supplier<Object>) HDBBuildingDbMgr::getInstance),
                Arguments.of("HDBBuildingGeoJsonDownloader", (Supplier<Object>) HDBBuildingGeoJsonDownloader::getInstance),
                Arguments.of("HDBBuildingGeoJsonImporter", (Supplier<Object>) HDBBuildingGeoJsonImporter::getInstance),
                Arguments.of("HDBBuildingMgr", (Supplier<Object>) HDBBuildingMgr::getInstance),
                Arguments.of("HDBBuildingSQLCreator", (Supplier<Object>) HDBBuildingSQLCreator::getInstance),
                Arguments.of("HDBBuildingSunFacingAnalysis", (Supplier<Object>) HDBBuildingSunFacingAnalysis::getInstance),
```

PROBLEMS 4 OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

```
[INFO] Scanning for projects...
[INFO] -------------------------------------------------------
[INFO] Running habitathero.GeoSpatialAnalysis.src.TestSingleton
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0, Time elapsed: 0.296 s -- in habitathero.GeoSpatialAnalysis.src.TestSingleton
[INFO]
[INFO] Results:
[INFO]
[INFO] Tests run: 30, Failures: 0, Errors: 0, Skipped: 0
[INFO]
[INFO] ------------------------------------------------------------------------
[INFO] BUILD SUCCESS
[INFO] ------------------------------------------------------------------------
[INFO] Total time:  10.854 s
```

PERSONA #1

PROFILE

- Retired couple whose children have moved out
- Looking to downsize to a 2-Room HDB flat
- Prefer mature estates (e.g. Ang Mo Kio, Bishan, Toa Payoh)

ACCESSIBILITY

Must be within walking distance to nearby hawker centres and supermarkets.

COMFORT

Must avoid strong west afternoon sun with minimal MRT/LRT noise.

BUDGET

Must be affordable to support long-term retirement living.

PERSONA #2

PROFILE

- Working couple with one partner commuting to Jurong and the other to the CBD
- Both have stable incomes and a relatively high housing budget
- Main priority is finding a home that is fair and efficient for both daily commutes

ACCESSIBILITY

Must support two-commuter travel analysis and provide convenient access to MRT stations or major transport routes for both Jurong and CBD travel.

COMFORT

Must minimize daily commute burden, avoid excessively long travel times, and provide a practical location for weekday work-life balance.

BUDGET

High budget with flexibility for a better-located flat,
as long as it reduces total commute time and improves travel fairness for both partners.

PERSONA #3

PROFILE

- Young family with a 5-year-old child
- Looking for a 5-Room HDB flat with enough space for a growing child
- Top priority is to stay in the East while staying near a primary school

ACCESSIBILITY

Must be close to the child's school and within convenient reach of supermarkets, schools, and playgrounds for daily family needs.

COMFORT

Must avoid strong west afternoon sun.
Acoustic comfort is important, but slightly flexible.

BUDGET

Must be affordable for a growing family while supporting the costs of a larger flat and long-term child-raising expenses.

SYSTEM ROBUSTNESS & COMPLIANCE

Thank you!